# JOBSHEET 11

# DYNAMIC ROUTING & STATIC GENERATION

# PEMROGRAMAN BERBASIS FRAMEWORK

**REIKA AMALIA SYAHPUTRI**

**2341720173**

**TI-3E / 19**

**TEKNOLOGI INFORMASI**

# Github:

[https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_11](https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_11)

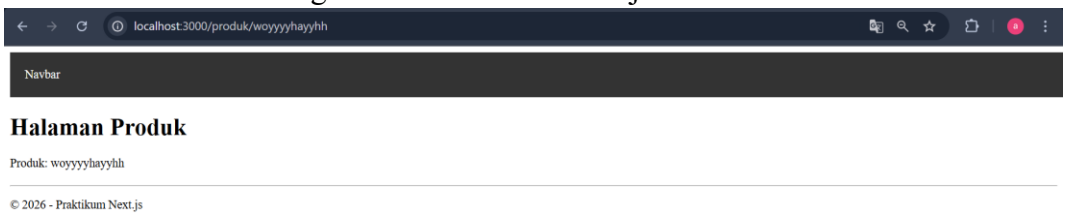
# C. Langkah Praktikum

### Bagian 1 – Membuat Dynamic Route

- Buka file src/views/produk/index.tsx dan modfikasi sbb ( line 30 )

```
1  import styles from "../../styles/product.module.scss";
2  import Link from "next/link";
3
4  type ProductType = {
5    id: string;
6    name: string;
7    price: number;
8    image: string;
9    category: string;
10 };
11
12 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
13   return (
14     <div className={styles.produk}>
15       <h1 className={styles.produk__title}>Daftar Produk</h1>
16
17       <div className={styles.produk__content}>
18         {products.length > 0 ? (
19
20           <>
21           {products.map((product: ProductType) => (
22             <Link href={`/produk/${product.id}`} key={product.id} className={styles.produk__content__item}>
23               <div
24                 // key={product.id}
25                 // className={styles.produk__content__item}
26               >
27                 <div className={styles.produk__content__item__image}>
28                   <img
29                     src={product.image}
30                     alt={product.name}
31                     width={200}
32                   />
33                 </div>
34
35                 <h4 className={styles.produk__content__item__name}>
36                   {product.name}
37                 </h4>
38
39                 <p className={styles.produk__content__item__category}>
40                   {product.category}
41                 </p>
42
43                 <p className={styles.produk__content__item__price}>
44                   Rp {product.price.toLocaleString("id-ID")}
45                 </p>
46               </div>
47             </Link>
48           ))}
49         </>
50
51         ) : (
52           <div className={styles.produk__content__skeleton}>
53               <div className={styles.produk__content__skeleton__item__image}> </div>
54               <div className={styles.produk__content__skeleton__item__name}> </div>
55               <div className={styles.produk__content__skeleton__item__category}> </div>
56               <div className={styles.produk__content__skeleton__item__price}> </div>
57             </div>
58         )}
59       </div>
60     </div>
61   );
62 };
63
64 export default TampilanProduk;
```

- Jalankan browser http://localhost:3000/produk
  Jika kita klik salah satu gambar maka akan menuju halaman lain

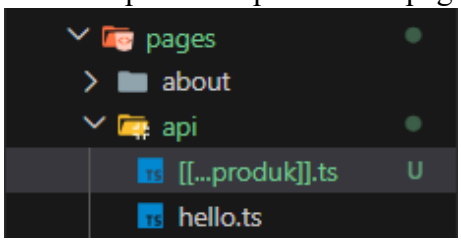



## Bagian 2 – Implementasi CSR (Client Rendering)

- Modifikasi pada file [produk].tsx pada folder src/pages/produk/

```
[produk].tsx U
my-app-11 > src > pages > produk > [produk].tsx > default
1  import { useRouter } from "next/router";
2  import useSWR from "swr";
3  import fetcher from "../../utils/swr/fetcher";
4
   Windsurf: Refactor | Explain | Generate JSDoc |
5  const HalamanProduk = () => {
6    // const Router = useRouter();
7    // console.log(Router);
8    const { query } = useRouter();
9    const{data, error, isLoading} = useSWR(`/api/produk/${query.id}`, fetcher);
10
11   return (
12     <div>
13       <h1>Halaman Produk</h1>
14       <p>Produk: {query.produk}</p>
15     </div>
16   );
17 };
18
19 export default HalamanProduk;
20
```

- Pada file produk.ts pada folder pages/api di rename menjadi [[...product]].ts




- Modifikasi file servicefirebase.ts

```
[produk].tsx U    servicefirebase.ts M ×
my-app-11 > src > utils > db > servicefirebase.ts > retrieveDataByID
import { getFirestore, collection, getDocs, Firestore, getDoc, doc } from "firebase/firestore";
import app from "./firebase";

const db = getFirestore(app);

Windsurf: Refactor | Explain | Generate JSDoc | ×
export async function retrieveProducts(collectionName: string) {
  const snapshot = await getDocs(collection(db, collectionName));
  const data = snapshot.docs.map((doc) => ({
    id: doc.id,
    ...doc.data(),
  }));
  return data;
}

Windsurf: Refactor | Explain | Generate JSDoc | ×
export async function retrieveDataByID(collectionName: string, id: string) {
  const snapshot = await getDoc(doc(db, collectionName, id));
  const data = snapshot.data();
  return data;
}
```

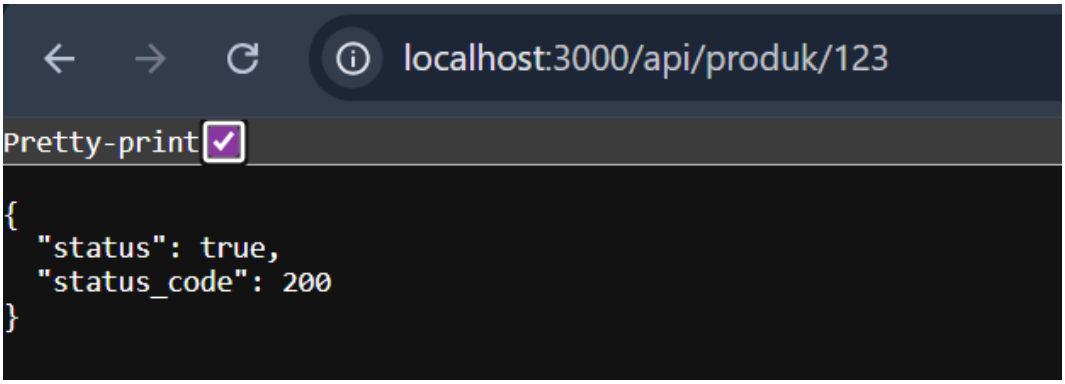
- Modifikasi file [[...produk]].ts

```
[produk].tsx U    servicefirebase.ts M    [[...produk]].ts U ×
my-app-11 > src > pages > api > [[...produk]].ts > ...
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";


import {
  retrieveDataByID,
  retrieveProducts,

} from "../../utils/db/servicefirebase";

Windsurf: Refactor | Explain
type Data = {
  status: boolean;
  status_code: number;
  data: any;
};

Windsurf: Refactor | Explain | Generate JSDoc | ×
export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>
) {
  if (req.query.produk?.[1]) {
    const data = await retrieveDataByID("products", req.query.produk![1]);
    res.status(200).json({ status: true, status_code: 200, data });
    return;
  } else {
    const data = await retrieveProducts("products");
    res.status(200).json({ status: true, status_code: 200, data });
  };
}
```

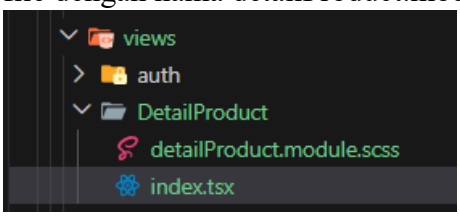
- Jalankan browser http://localhost:3000/api/produk/3FwB5UeuLVBpQy2umXHD

```
localhost:3000/api/produk/hviDGTeFjhUU3IUpLa9g
Pretty-print ☑
{
  "status": true,
  "status_code": 200,
  "data": {
    "price": 2000000,
    "name": "nike p-6000",
    "image": "https://static.nike.com/a/images/t_web_pdp_535_v2/f_auto,u_9ddf04c7-2a9a-4d76-add1-d15af8f0263d,c_scale,fl_relative,w_1.0,h_1.0,fl_layer_apply/873a17ee-d4be-49e2-804b-
88be84bab00e/W+NIKE+P-6000.png",
    "category": "sepatu"
  }
}
```

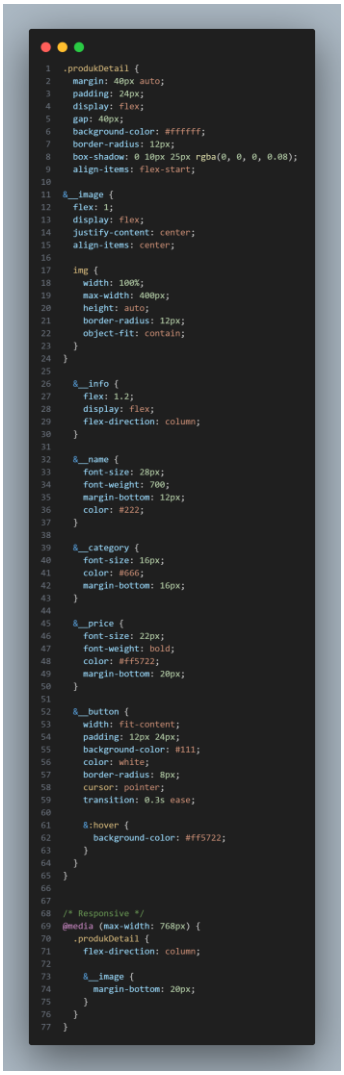
- Jalankan alamat url http://localhost:3000/api/produk/123




- Buat file dengan nama index.tsx pada folder views/DetailProduct selain itu buat juga file dengan nama detailProduct.module.scss




- Modifikasi detailProduct.module.scss



```scss
.produkDetail {
  margin: 40px auto;
  padding: 24px;
  display: flex;
  gap: 40px;
  background-color: #ffffff;
  border-radius: 12px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.08);
  align-items: flex-start;

  &__image {
    flex: 1;
    display: flex;
    justify-content: center;
    align-items: center;

    img {
      width: 100%;
      max-width: 400px;
      height: auto;
      border-radius: 12px;
      object-fit: contain;
    }
  }

  &__info {
    flex: 1.2;
    display: flex;
    flex-direction: column;
  }

  &__name {
    font-size: 28px;
    font-weight: 700;
    margin-bottom: 12px;
    color: #222;
  }

  &__category {
    font-size: 16px;
    color: #666;
    margin-bottom: 16px;
  }

  &__price {
    font-size: 22px;
    font-weight: bold;
    color: #ff5722;
    margin-bottom: 20px;
  }

  &__button {
    width: fit-content;
    padding: 12px 24px;
    background-color: #111;
    color: white;
    border-radius: 8px;
    cursor: pointer;
    transition: 0.3s ease;

    &:hover {
      background-color: #ff5722;
    }
  }
}


/* Responsive */
@media (max-width: 768px) {
  .produkDetail {
    flex-direction: column;

    &__image {
      margin-bottom: 20px;
    }
  }
}
```

- Modifikasi index.tsx pada folder DetailProduct

```
index.tsx U ×   servicefirebase.ts M   [[...produk]].ts U   product.type.ts   detailProduct.modu
my-app-11 > src > views > DetailProduct > index.tsx > default
1   import { ProductType } from "../../types/product.type";
2   import styles from "../DetailProduct/detailProduct.module.scss";
3
    Windsurf: Refactor | Explain | Generate JSDoc | ×
4   const DetailProduk = ({ products }: { products: ProductType }) => {
5     return (
6       <div className={styles.produkdetail}>
7         <div className={styles.produkdetail__image}>
8           <img src={products.image} alt={products.name} />
9         </div>
10
11        <div className={styles.produkdetail__info}>
12          <h1 className={styles.produkdetail__name}>{products.name}</h1>
13          <p className={styles.produkdetail__category}>{products.category}</p>
14          <p className={styles.produkdetail__price}>
15            Rp {products.price.toLocaleString("id-ID")}
16          </p>
17        </div>
18      </div>
19    );
20  };
21
22  export default DetailProduk;
```

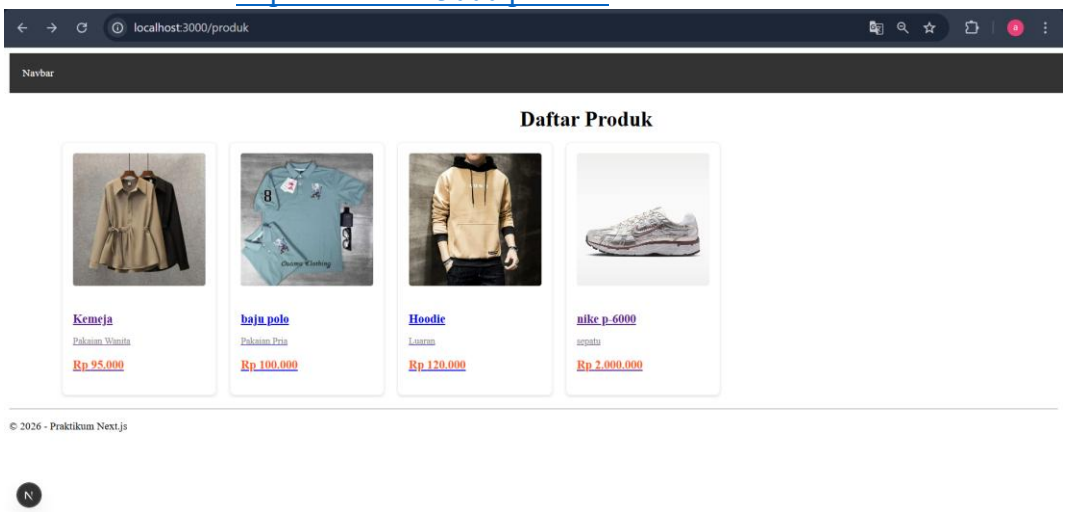
- Modifikasi file [product].tsx

```
[produk].tsx U ×   index.tsx U   servicefirebase.ts M   [[...produk]].ts U   product.type.ts
my-app-11 > src > pages > produk > [produk].tsx > HalamanProduk
1   import { useRouter } from "next/router";
2   import useSWR from "swr";
3   import fetcher from "../../utils/swr/fetcher";
4   import DetailProduk from "@/views/DetailProduct";
5
    Windsurf: Refactor | Explain | Generate JSDoc | ×
6   const HalamanProduk = () => {
7     // const Router = useRouter();
8     // console.log(Router);
9     const { query } = useRouter();
10    const{data, error, isLoading} = useSWR(`/api/produk/${query.produk}`, fetcher);
11
12    return (
13      <div>
14        <DetailProduk products={isLoading ? [] :data.data} />
15      </div>
16    );
17  };
18
19  export default HalamanProduk;
```

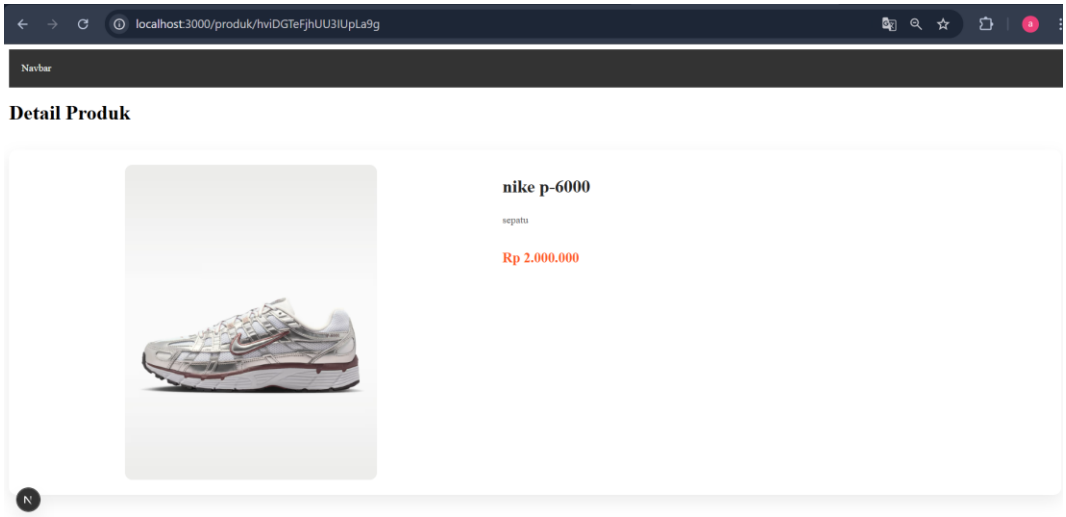
- Modifikasi index.tsx pada folder views/detailProduct line 16

```
[produk].tsx U   index.tsx U ×   servicefirebase.ts M   [[...produk]].ts U   product.type.ts
my-app-11 > src > views > DetailProduct > index.tsx > ...
1   import { ProductType } from "../../types/product.type";
2   import styles from "../DetailProduct/detailProduct.module.scss";
3
    Windsurf: Refactor | Explain | Generate JSDoc | ×
4   const DetailProduk = ({ products }: { products: ProductType }) => {
5     return (
6       <>
7       <h1>Detail Produk</h1>
8       <div className={styles.produkdetail}>
9         <div className={styles.produkdetail__image}>
10          <img src={products.image} alt={products.name} />
11        </div>
12
13        <div className={styles.produkdetail__info}>
14          <h1 className={styles.produkdetail__name}>{products.name}</h1>
15          <p className={styles.produkdetail__category}>{products.category}</p>
16          <p className={styles.produkdetail__price}>
17            Rp {products.price.toLocaleString("id-ID")}
18          </p>
19        </div>
20      </div>
21      </>
22    );
23  };
24
25  export default DetailProduk;
```

- Jalankan browser http://localhost:3000/produk/




saat produk diklik maka akan muncul detailProduk http://localhost:3000/produk/pAWIT99SWmVbVrNm49ml



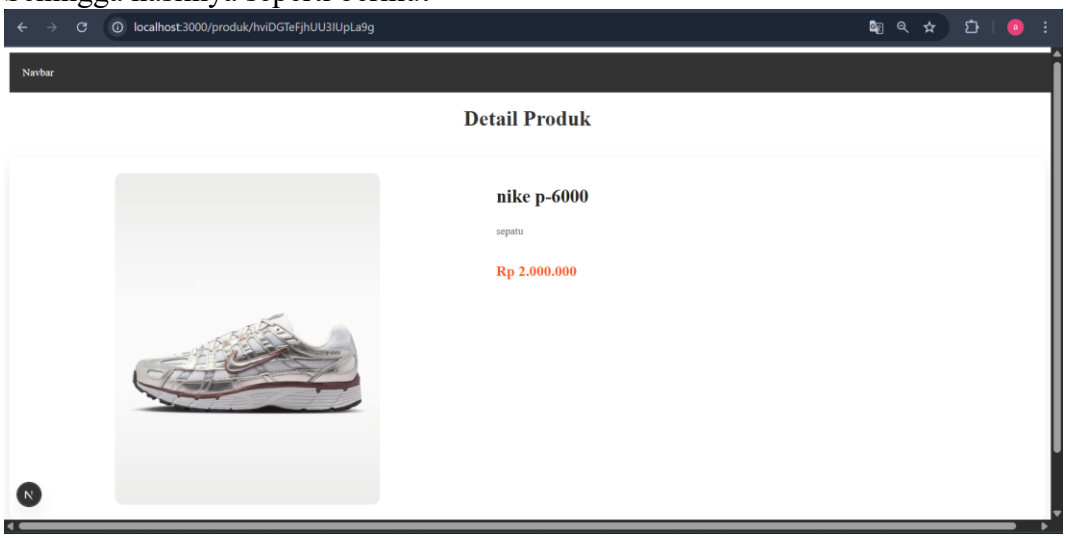

- Agar tulisan detail produk ditengah maka modifikasi file detailProduct.module.scss line 103-108

```
1   &__title {
2     text-align: center;
3     font-size: 32px;
4     font-weight: bold;
5     margin-bottom: 40px;
6     color: #333;
7   }
8 }
```

dan file index.tsx tambahkan code pada line 7,8 dan 22 menjadi

```
import { ProductType } from "../../types/product.type";
import styles from "../DetailProduct/detailProduct.module.scss";

const DetailProduk = ({ products }: { products: ProductType }) => {
  return (
    <>
    <h1 className={styles.produkDetail__title}>Detail Produk</h1>
    <div className={styles.produkDetail}>
      <div className={styles.produkDetail__image}>
        <img src={products.image} alt={products.name} />
      </div>

      <div className={styles.produkDetail__info}>
        <h1 className={styles.produkDetail__name}>{products.name}</h1>
        <p className={styles.produkDetail__category}>{products.category}</p>
        <p className={styles.produkDetail__price}>
          Rp {products.price && products.price.toLocaleString("id-ID")}
        </p>
      </div>
    </div>
    </>
  );
};

export default DetailProduk;
```

- Sehingga hasilnya seperti berikut

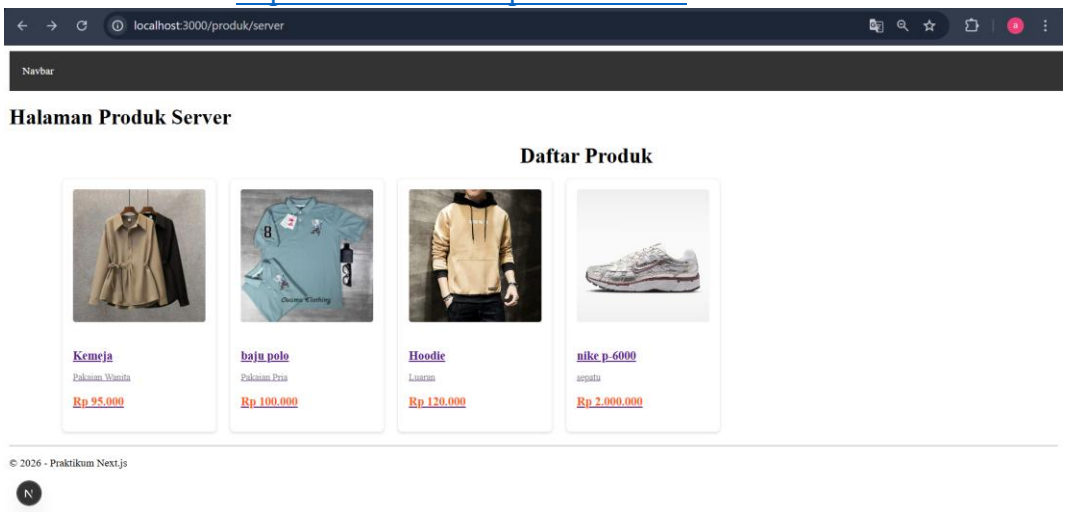


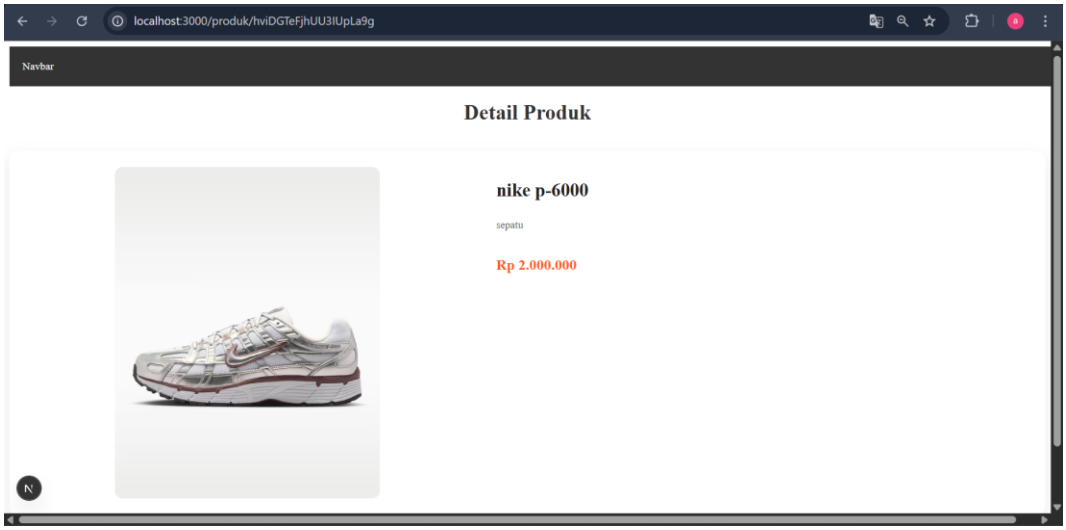
## Bagian 3 – Implementasi SSR

- Modifikasi [produk].tsx pada folder src/pages/produk dan comment line 9 sampai 20 dikarena kita akan menggunakan metode SSR. Tambahkan beberapa kode untuk SSR

```
1  import { useRouter } from "next/router";
2  import useSWR from "swr";
3  import fetcher from "../../utils/swr/fetcher";
4  import DetailProduk from "@/views/DetailProduct";
5  import { ProductType } from "../../types/product.type";
6
7  const HalamanProduk = ({ product }: { product: ProductType }) => {
8    // const Router = useRouter();
9    // console.log(Router);
10   // const { query } = useRouter();
11   // const{data, error, isLoading} = useSWR(`/api/produk/${query.produk}`, fetcher);
12
13   // return (
14   //    <div>
15   //     <DetailProduk products={isLoading ? [] :data.data} />
16   //    </div>
17   // );
18
19
20   return (
21     <div>
22       <DetailProduk products={product} />
23     </div>
24   )
25 };
26
27 export default HalamanProduk;
28
29 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
30 // {digunakan server-side rendering}
31 export async function getServerSideProps({ params }: { params: { produk: string } }) {
32   const res = await fetch(`http://localhost:3000/api/produk/${params?.produk}`);
33   const response = await res.json();
34   // console.log("Data produk yang diambil dari API:", response);
35   return {
36     props: {
37       product: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
38     },
39   };
40 }
```

- Jalankan browser http://localhost:3000/produk/server






## Bagian 4 – Implementasi Static Site Generation (Dynamic SSG)

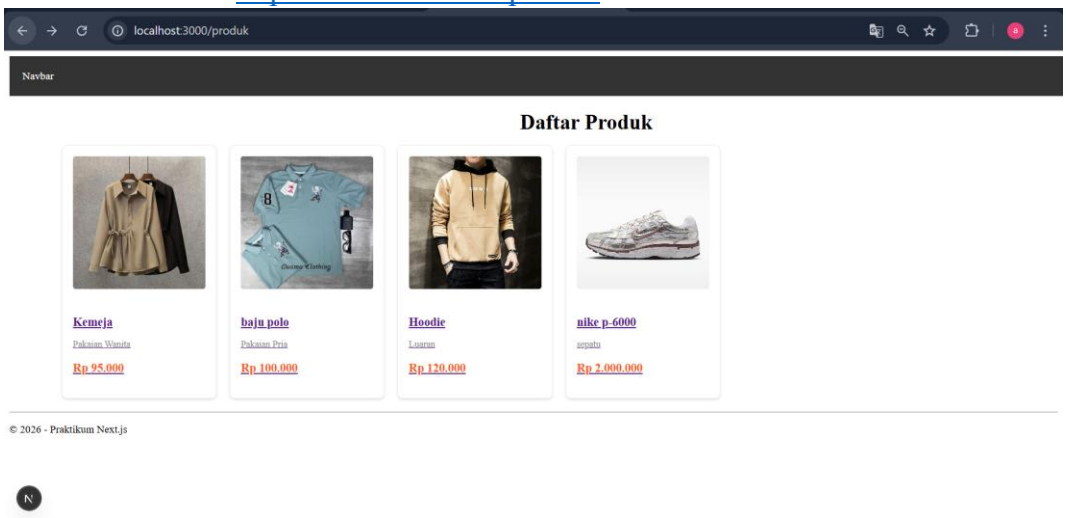
- Buka file [produk].tsx dan modifikasi seperti berikut

```
import { useRouter } from "next/router";
import useSWR from "swr";
import fetcher from "../../utils/swr/fetcher";
import DetailProduk from "@/views/DetailProduct";
import { ProductType } from "../../types/product.type";

const HalamanProduk = ({ product }: { product: ProductType }) => {
  {/digunakan client-side rendering/}
  // const Router = useRouter();
  // console.log(Router);
  // const { query } = useRouter();
  // const{data, error, isloading} = useSWR(`/api/produk/${query.produk}`, fetcher);

  // return (
  //   <div>
  //     <DetailProduk products={isLoading ? [] :data.data} />
  //   </div>
  // );


  return (
    <div>
      <DetailProduk products={product} />
    </div>
  )
};

export default HalamanProduk;

{/digunakan client-side rendering/}
// //export async function getServerSideProps({ params }: { params: { produk: string } }) {
// //   const res = await fetch(`http://localhost:3000/api/produk/${params?.produk}`);
// //   const respone = await res.json();
// //   // console.log("Data produk yang diambil dari API:", respone);
// //   return {
// //     props: {
// //       product: respone.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
// //     },
// //   };
// // }



// Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
// {digunakan server-side rendering}


{/digunakan static-site generation/}
export async function getStaticPaths() {
  const res = await fetch('http://localhost:3000/api/products')
  const response = await res.json()

  const paths = response.data.map((product: ProductType) => ({
    params: { produk: product.id }
  }));
  // console.log("Paths yang dihasilkan untuk produk:", paths); // Debugging: Tampilkan paths yang dihasilkan
  return {
    paths,
    fallback: false
  }
}

export async function getStaticProps({params}:{params:{produk:string}}) {
  const res = await fetch(`http://localhost:3000/api/produk/${params?.produk}`);
  // const response: ProductType[] = await res.json();
  const response: { data: ProductType[] } = await res.json();

  // console.log("Data produk yang diambil dari API:", response);
  return {
    props: {
      product: response.data,
    }
  }
}

```

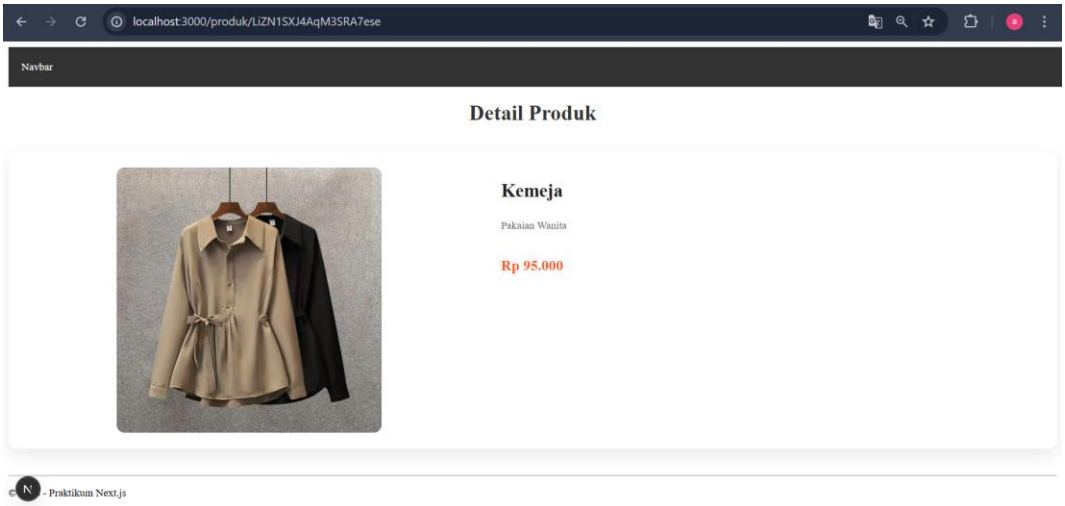
- Buka file index.tsx pada folder src/views/DetailProduct dan modifikasi pada line 11

```
1  import { ProductType } from "../../types/product.type";
2  import styles from "../DetailProduct/detailProduct.module.scss";
3
4  const DetailProduk = ({ products }: { products: ProductType }) => {
5    return (
6      <>
7      <h1 className={styles.produkDetail__title}>Detail Produk</h1>
8      <div className={styles.produkDetail}>
9        <div className={styles.produkDetail__image}>
10         <img src={products.image && products.image} alt={products.name} />
11       </div>
12
13       <div className={styles.produkDetail__info}>
14         <h1 className={styles.produkDetail__name}>{products.name}</h1>
15         <p className={styles.produkDetail__category}>{products.category}</p>
16         <p className={styles.produkDetail__price}>
17           Rp {products.price && products.price.toLocaleString("id-ID")}
18         </p>
19       </div>
20     </div>
21     </>
22   );
23 };
24
25 export default DetailProduk;
```

- Jalankan browser http://localhost:3000/produk



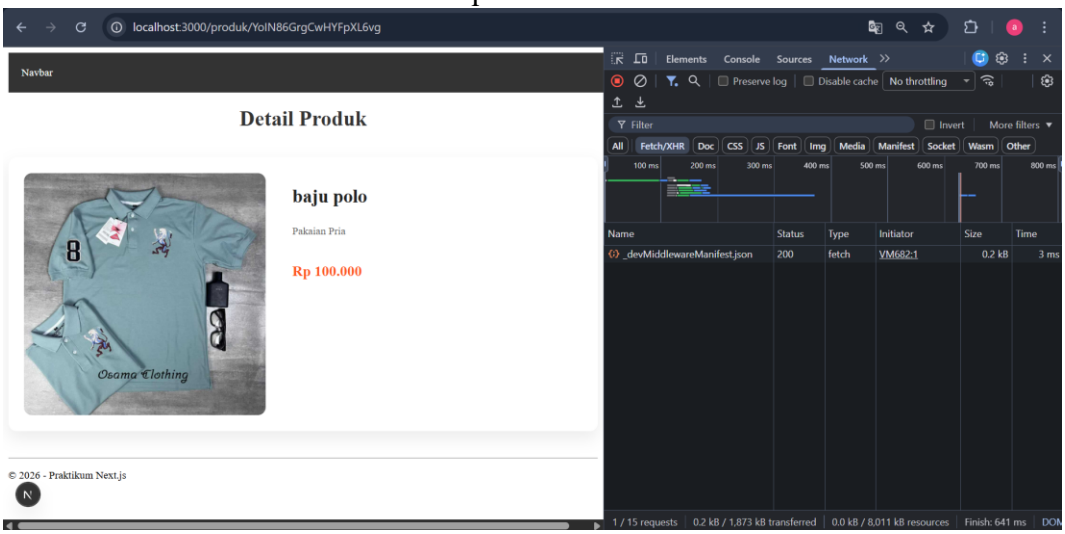

- Saat diklik salah satu produk




## D. Pengujian

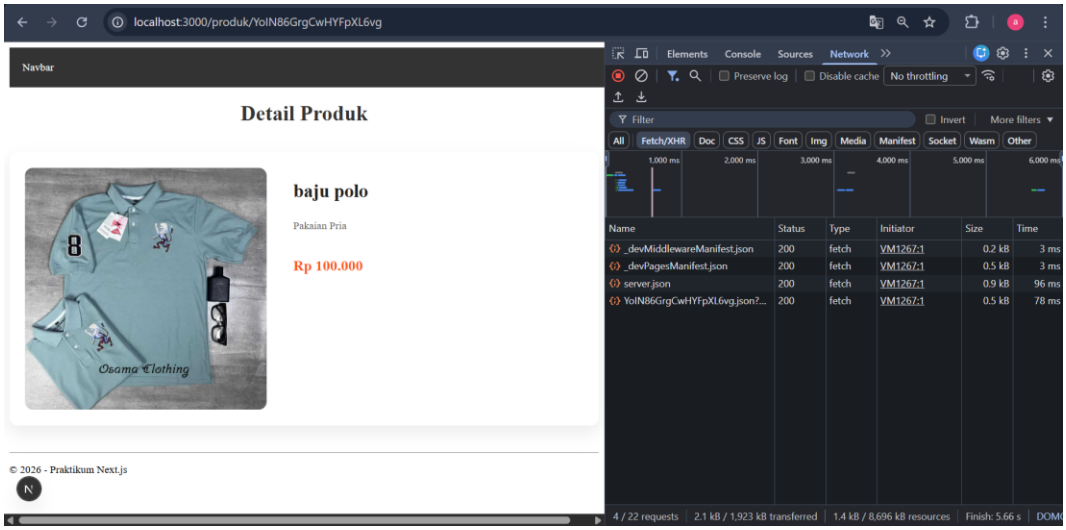
### Uji 1 – CSR

- Refresh halaman detail
- Perhatikan loading
- Periksa Network → XHR → API request terlihat




### Uji 2 – SSR

- Refresh halaman detail
- Tidak ada loading
- Periksa Network → tidak terlihat fetch detail

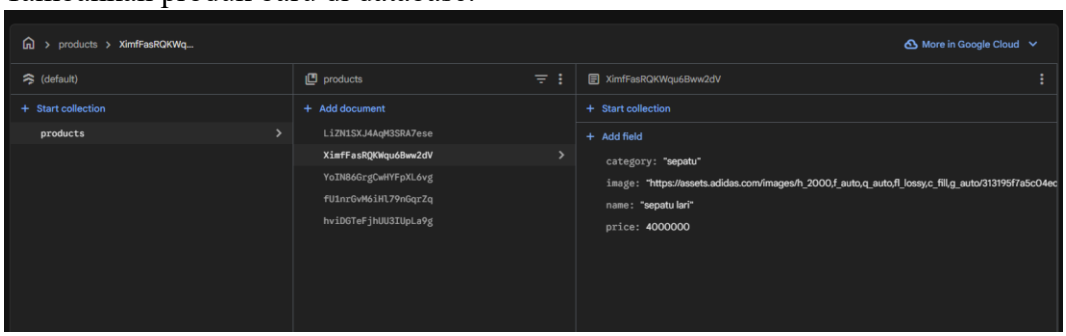



**Uji 3 – SSG ( Lakukan seperti langkah sebelumya pada Jobsheet 10)**

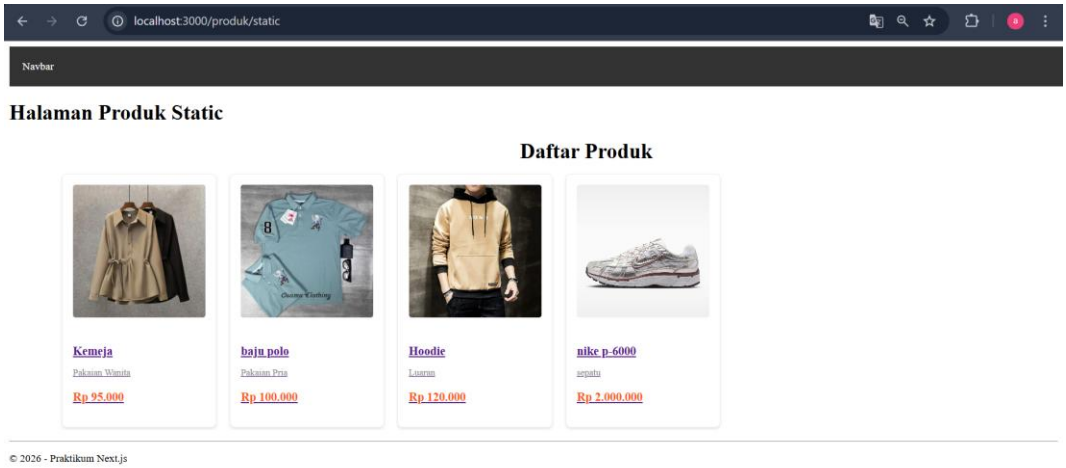
1. Jalankan:

- npm run build
- npm run start

2. Tambahkan produk baru di database.




3. Buka halaman detail produk baru:
   Tidak muncul.

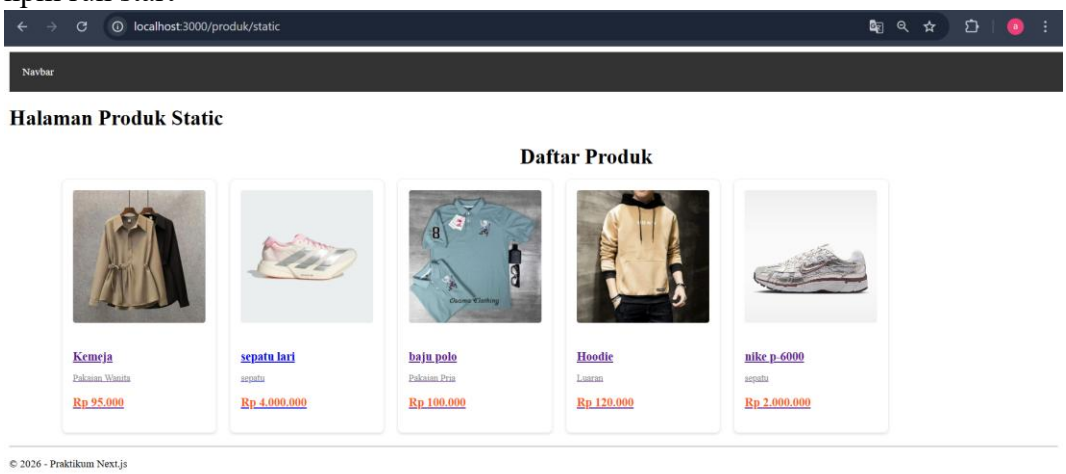


4. Build ulang:

- npm run build

```
PS D:\Pemograman_Framework\praktikum_11\my-app-11> npm run build
  │ ├ /produk/LiZN1SXJ4AqM3SRA7ese
  │ ├ /produk/XimfFasRQKWqu6Bww2dV
  │ ├ /produk/YoIN86GrgCwHYFpXL6vg
  │ └ [+2 more paths]
  ├ ƒ /produk/server
  ├ ● /produk/static
  ├ ○ /profile
  ├ ○ /profile/edit
  ├ ○ /setting/app
  ├ ○ /shop/[[...slug]]
  ├ ○ /user
  └ ○ /user/password

○  (Static)   prerendered as static content
●  (SSG)      prerendered as static HTML (uses getStaticProps)
ƒ  (Dynamic)  server-rendered on demand
```

- npm run start

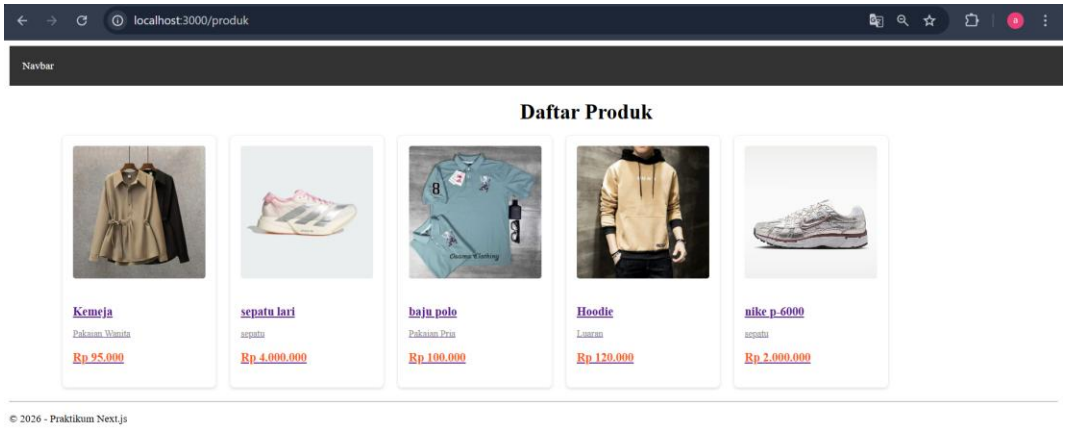


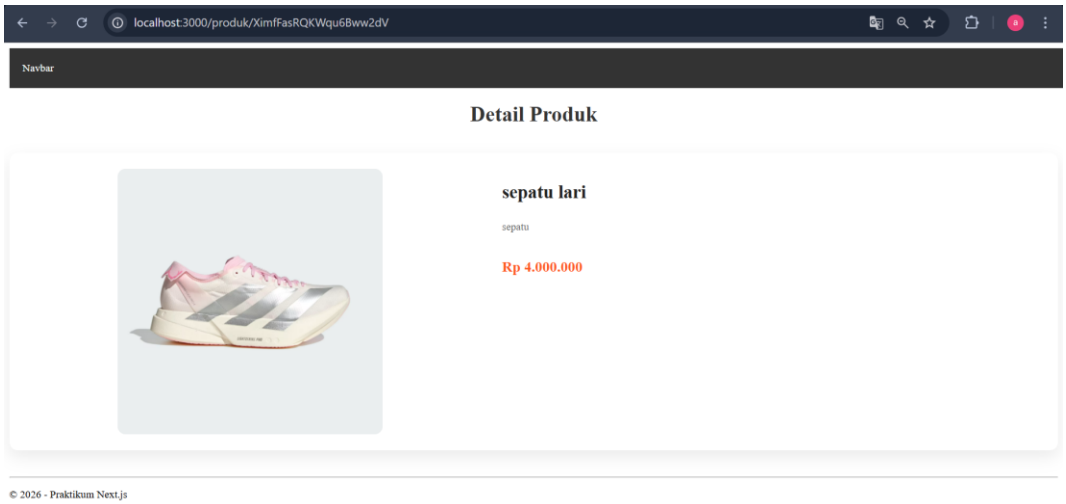

# E. Tugas Praktikum

## Tugas Individu

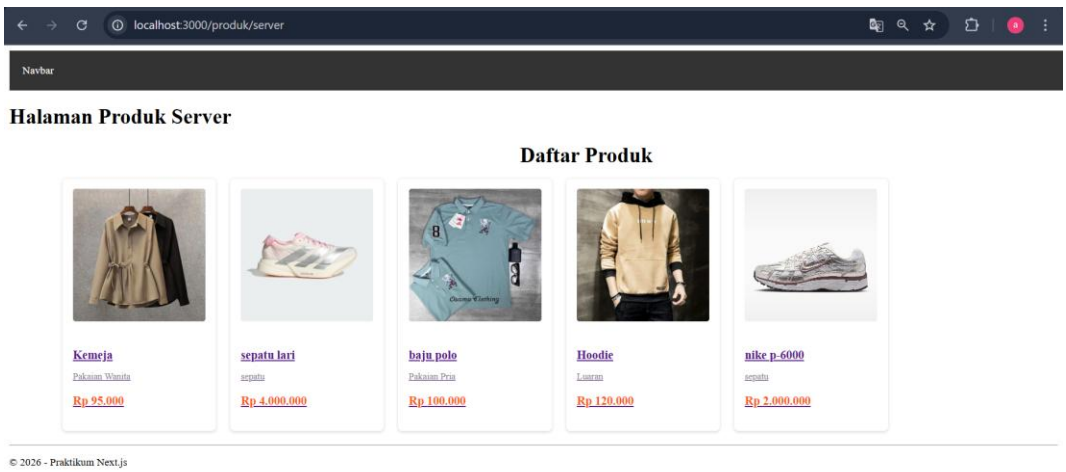
1. Implementasikan halaman detail dengan:

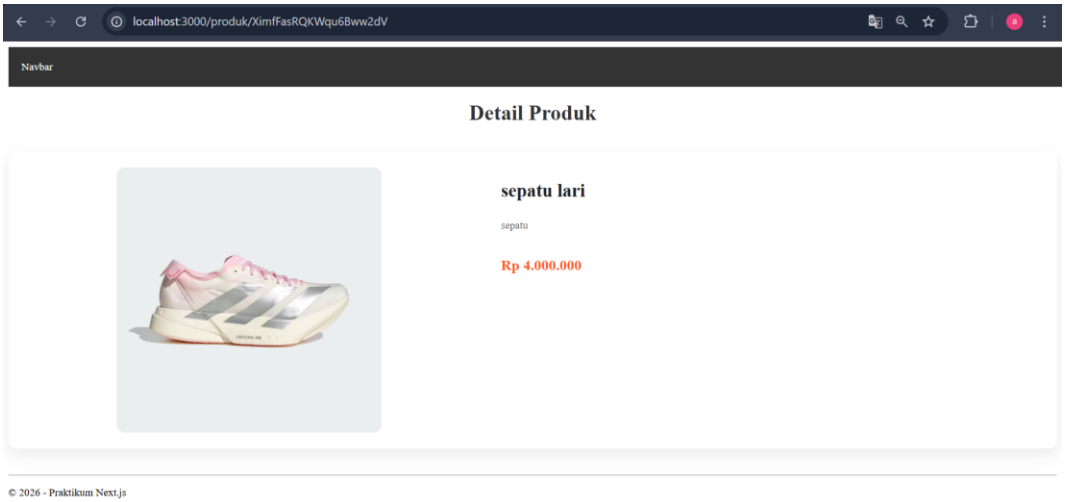
- CSR


localhost:3000/produk
Navbar
Daftar Produk
Kemeja
Pakaian Wanita
Rp 95.000
sepatu lari
sepatu
Rp 4.000.000
baju polo
Pakaian Pria
Rp 100.000
Hoodie
Luaran
Rp 120.000
nike p-6000
sepatu
Rp 2.000.000
© 2026 - Praktikum Next.js

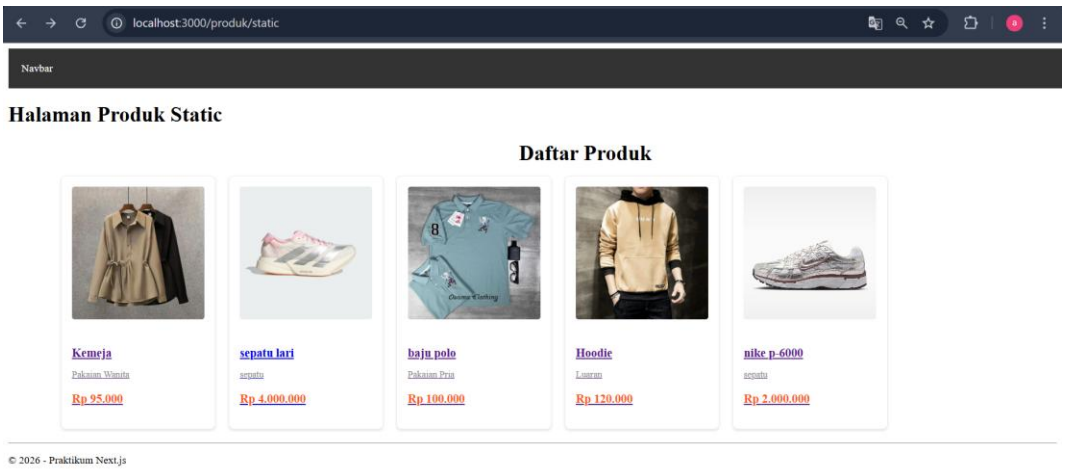

localhost:3000/produk/XimfFasRQKWqu6Bww2dV
Navbar
Detail Produk
sepatu lari
sepatu
Rp 4.000.000
© 2026 - Praktikum Next.js

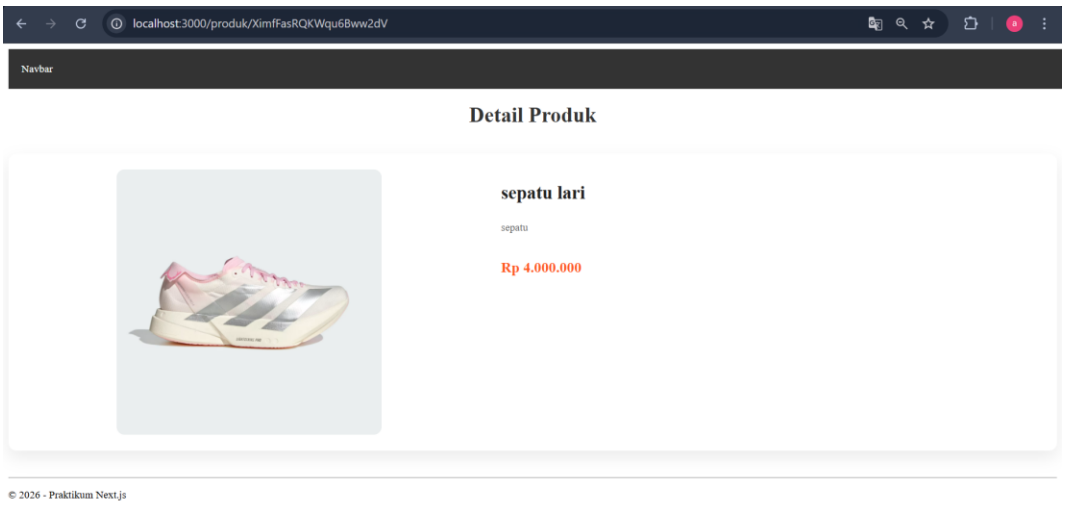
- SSR


localhost:3000/produk/server
Navbar
Halaman Produk Server
Daftar Produk
Kemeja
Pakaian Wanita
Rp 95.000
sepatu lari
sepatu
Rp 4.000.000
baju polo
Pakaian Pria
Rp 100.000
Hoodie
Luaran
Rp 120.000
nike p-6000
sepatu
Rp 2.000.000
© 2026 - Praktikum Next.js


localhost:3000/produk/XimfFasRQKWqu6Bww2dV
Navbar
Detail Produk
sepatu lari
sepatu
Rp 4.000.000
© 2026 - Praktikum Next.js

- SSG


localhost:3000/produk/static
Navbar
Halaman Produk Static
Daftar Produk
Kemeja
Pakaian Wanita
Rp 95.000
sepatu lari
sepatu
Rp 4.000.000
baju polo
Pakaian Pria
Rp 100.000
Hoodie
Luaran
Rp 120.000
nike p-6000
sepatu
Rp 2.000.000
© 2026 - Praktikum Next.js


localhost:3000/produk/XimfFasRQKWqu6Bww2dV
Navbar
Detail Produk
sepatu lari
sepatu
Rp 4.000.000
© 2026 - Praktikum Next.js

2. Buat tabel perbandingan

| Aspek | CSR | SSR | SSG |
|---|---|---|---|
| Loading | Loading lebih lama di awal karena data diambil di browser setelah halaman dimuat. | Loading cukup cepat karena data sudah diambil di server sebelum halaman dikirim ke browser. | Loading paling cepat karena halaman sudah dibuat saat proses build. |
| Build Required | Tidak memerlukan build untuk mengambil data karena data diambil saat halaman dibuka. | Tidak memerlukan build khusus karena data diambil setiap kali ada request ke server. | Memerlukan proses build karena halaman dibuat terlebih dahulu saat build. |
| SEO | Kurang baik untuk SEO karena konten dimuat setelah JavaScript dijalankan di browser. | Baik untuk SEO karena konten sudah tersedia saat halaman dikirim dari server. | Sangat baik untuk SEO karena halaman sudah berupa HTML statis saat diakses. |
| Perubahan Data | Data selalu terbaru karena diambil langsung dari API saat halaman dibuka. | Data selalu terbaru karena server mengambil data setiap ada request. | Data tidak langsung berubah karena halaman dibuat saat build, sehingga perlu build ulang untuk memperbarui data. |

# F. Pertanyaan Analisis

1. Mengapa getStaticPaths wajib pada dynamic SSG?
- Pada dynamic SSG, Next.js perlu mengetahui halaman apa saja yang harus dibuat saat proses build. Fungsi getStaticPaths digunakan untuk mengambil daftar parameter atau id dari data yang ada, misalnya id produk. Dengan begitu Next.js dapat membuat halaman statis untuk setiap parameter tersebut pada saat build berlangsung.

2. Mengapa CSR membutuhkan loading state?
- Pada metode CSR, data diambil dari API setelah halaman dimuat di browser. Karena proses pengambilan data membutuhkan waktu, halaman biasanya belum memiliki data saat pertama kali ditampilkan. Oleh karena itu diperlukan loading state untuk memberi informasi kepada pengguna bahwa data sedang dimuat.

3. Mengapa SSG tidak menampilkan produk baru tanpa build ulang?
- Pada SSG, halaman dibuat terlebih dahulu saat proses build dan kemudian disimpan sebagai file statis. Jika ada data baru setelah proses build selesai, halaman yang sudah dibuat tidak akan otomatis diperbarui. Oleh karena itu diperlukan build ulang agar halaman statis dapat memuat data terbaru.

4. Mana metode terbaik untuk halaman detail e-commerce?

- Untuk halaman detail produk e-commerce, metode yang sering digunakan adalah SSR. Hal ini karena data produk dapat berubah sewaktu-waktu, seperti harga atau stok. Dengan SSR, data akan diambil setiap kali halaman diakses sehingga informasi yang ditampilkan lebih terbaru.

5. Apa risiko menggunakan SSG untuk produk yang sering berubah?

- Risiko menggunakan SSG pada produk yang sering berubah adalah informasi yang ditampilkan bisa menjadi tidak sesuai dengan kondisi terbaru. Misalnya perubahan harga, stok, atau produk baru tidak langsung terlihat di halaman. Hal ini dapat menyebabkan pengguna mendapatkan informasi yang tidak akurat sebelum dilakukan build ulang aplikasi.